Artery Segmentation and Stenosis Detection Using Advanced Deep Learning Techniques

Milestone 1: Week 1 & 2 — Project Initialization, Dataset & Core Setup

1. Project Objectives

Main Objective

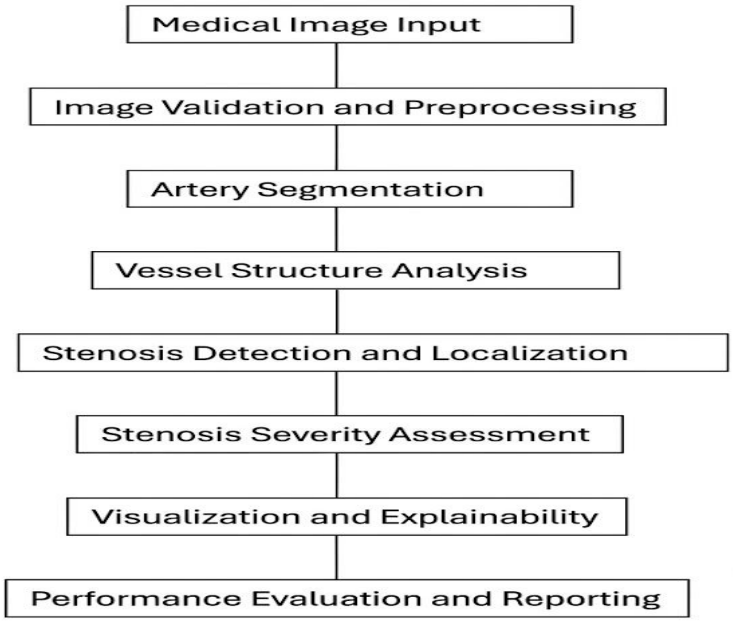
We aim to develop an Medical image analysis system that automatically segments arteries, identifies abnormal arterial regions, detects and localizes stenosis, and estimates the severity of arterial narrowing using advanced deep learning techniques. We intend to assist clinicians and medical imaging specialists by reducing manual analysis effort and providing quantitative measurements of arterial structures and stenosis.

Specific Objectives

- Automated artery segmentation and structure analysis
- Stenosis detection and localization
- Severity estimation
- Visualization
- Performance evaluation
- Application development

2. Proposed Project Workflow

We propose the following workflow for our project:



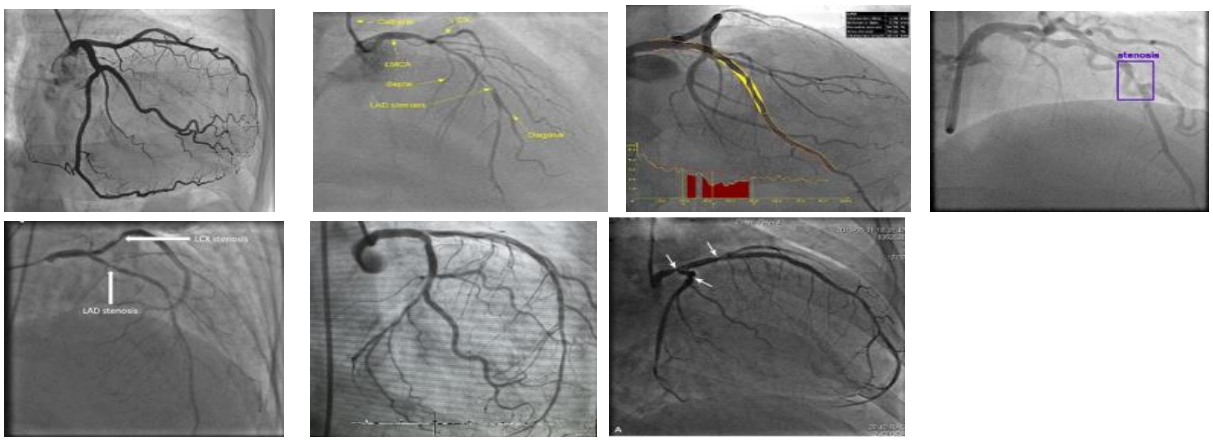
3. Scope of Milestone 1

During Weeks 1 and 2, we will focus on establishing the project foundation. We will:

- Finalize the target artery and imaging modality.
- Select and download a suitable public or de-identified dataset.
- Understand the dataset structure and annotations.
- Perform exploratory data analysis (EDA).
- Design the system architecture, database schema, and UI wireframes.
- Set up the frontend and backend.
- Implement authentication, case management, and medical image upload.
- Build the initial image preprocessing pipeline.

1. Select the medical imaging modality

Our proposed choice: 2D X-ray coronary angiography.



1. Dataset Selection and Understanding

We selected the ARCADE coronary angiography dataset for artery-related image analysis and stenosis segmentation. The dataset contains medical angiography images in PNG format. The inspected images have a resolution of 512 × 512 pixels. The dataset was downloaded, extracted into the project directory, and inspected to understand its organization and suitability for the planned deep learning pipeline.

The stenosis dataset is divided into three subsets:

Dataset split	Number of images
Training	1,000
Validation	200
Testing	300
Total	**1,500**

2. Dataset Structure Analysis

We examined the directory structure to identify the locations of images and their corresponding annotations. The dataset follows this organization:

- arcade/stenosis/train/images/ — training images

- arcade/stenosis/train/annotations/ — training annotations
- arcade/stenosis/val/images/ — validation images
- arcade/stenosis/val/annotations/ — validation annotations
- arcade/stenosis/test/images/ — testing images
- arcade/stenosis/test/annotations/ — testing annotations

3. Annotation Analysis

We inspected the JSON annotation files to understand their structure and how stenosis regions are represented. The annotation files contain three main components:

- **Images:** Image filenames, image IDs, width, and height.
- **Annotations:** Category IDs, segmentation of polygons, bounding boxes, and area information.
- **Categories:** Category IDs and their corresponding labels.

The stenosis category was identified as **category ID 26**, with the category name **stenosis**. The polygon coordinates describe the annotated regions within the images. These coordinates provide the information required to generate target masks for a segmentation model.

4. Exploratory Data Analysis

We performed initial exploratory data analysis (EDA) to understand the dataset and inspect its annotations. The completed activities included:

- Counting images and annotations across the training, validation, and testing.
- Examining category IDs and identifying the stenosis category.
- Inspecting image dimensions and annotation formats.
- Analyzing the distribution of stenosis annotations across the dataset.
- Identifying three training images without explicit stenosis annotations.
- Verifying that the corresponding image files existed.
- Visualizing annotation polygons over the original angiography images.

The analysis identified the following stenosis annotation counts:

Dataset split	Stenosis annotations
Training	1,625
Validation	406
Testing	386
Total	**2,417**

Three training images had no explicit stenosis annotation. These images were recorded for further investigation rather than automatically being classified as normal images.

5. Annotation Visualization

We created visualizations showing the original angiography images alongside their stenosis annotation overlays. This helped verify that polygon coordinates could be mapped onto the corresponding images and made the labeled regions easier to inspect. The visualization was used for structural inspection of the annotations. It does not establish the clinical correctness of the labels.

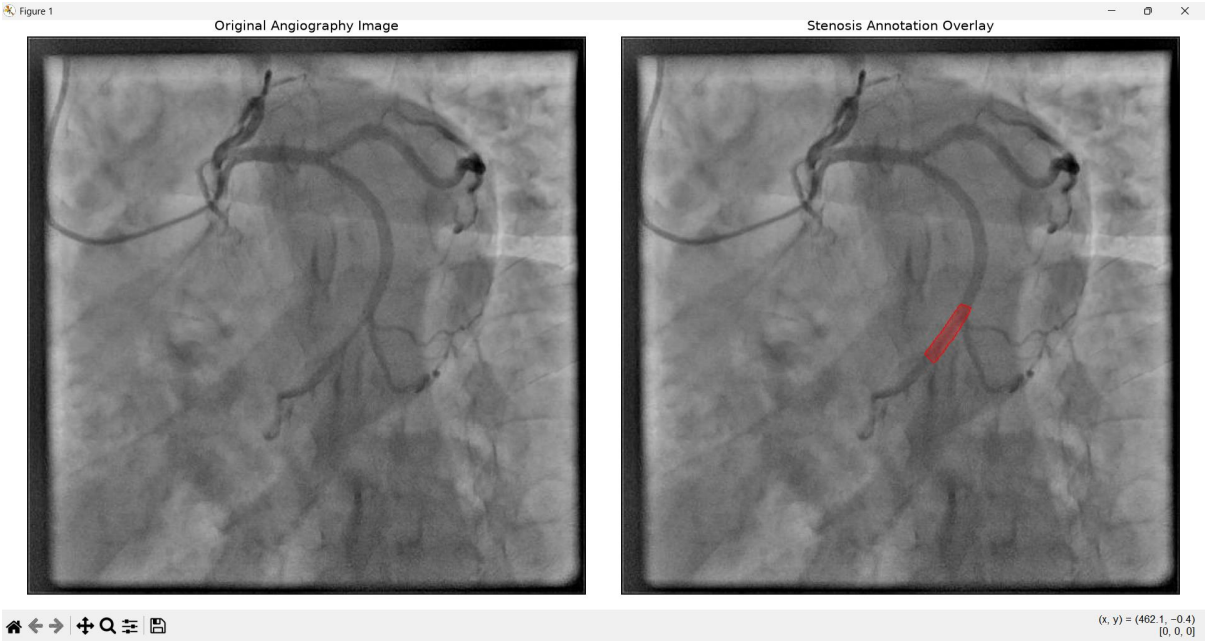


Figure 2. Visualization of stenosis annotation polygons on an original angiography image.

6. Binary Stenosis Mask Generation

We developed a Python script to convert stenosis polygon annotations into binary segmentation masks. The script processes the training, validation, and testing subsets separately. It selects annotations belonging to category ID 26 and rasterizes their polygon coordinates into mask images.

The generated masks use the following pixel values:

- **0:** Background or pixels outside the annotated stenosis region.
- **255:** Pixels inside the annotated stenosis region.

The masks are stored in the `processed/stenosis/` directory, organized by dataset split. A manifest CSV file is also generated for each split to record image and mask information.

The mask generation results are summarized below:

Dataset split	Original images	Generated masks	Images without stenosis annotations
Training	1,000	997	3
Validation	200	200	0
Testing	300	300	0
Total	**1,500**	**1,497**	**3**

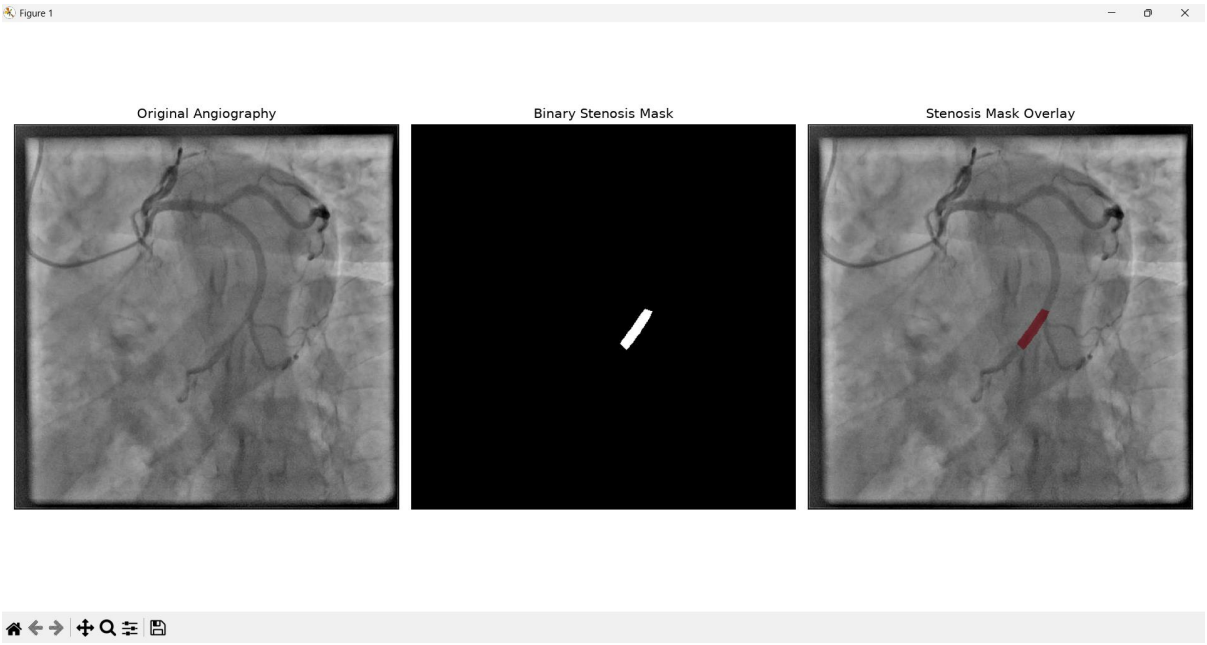


Figure 3. Example of binary stenosis mask generation from polygon annotations.

7. Data Quality Verification

We developed verification scripts to check the generated masks and their corresponding images. The checks included:

1. Verifying that original images and generated masks exist.
2. Checking that image and mask dimensions match.
3. Checking that masks contain only the expected binary values.
4. Checking annotation references for missing image IDs.
5. Checking polygon annotations for basic structural validity.

The verification scripts reported no missing image files or masks among the annotated image-mask pairs, no dimension mismatches, and no invalid mask pixel values. These checks establish basic structural consistency of the prepared data. They do not establish clinical validation or correctness of the annotations.

8. Image Preprocessing

We implemented and tested an initial preprocessing procedure using a sample training image and its corresponding mask.

The preprocessing steps include:

1. Loading the angiography image.
2. Converting the image to grayscale.
3. Normalizing image pixel values from the range 0–255 to the range 0–1.
4. Converting the mask from pixel values 0 and 255 into binary values 0 and 1.

5. Converting the image and mask arrays into the appropriate numerical data types for PyTorch.

The preprocessing test was successful. The image was represented as a `float32` array, while the binary mask used integer values before conversion into a tensor.

9. PyTorch Dataset Implementation

We created a Python file named `stenosis_dataset.py`, containing a custom PyTorch Dataset class called `StenosisDataset`.

The class performs the following operations:

1. Loads image and mask information from the manifest CSV file.
2. Select samples with stenosis annotations.
3. Loads the corresponding angiography image and binary mask.
4. Converts the image to grayscale and normalizes it.
5. Converts the mask into binary values.
6. Converts both arrays into PyTorch tensors.
7. Adds a channel dimension to prepare the data for a convolutional neural network.

The dataset loader was tested successfully using sample training images. The resulting image and mask tensors have the shape `(1, 512, 512)`, representing one channel and a spatial resolution of 512 × 512 pixels.

10. PyTorch DataLoader Implementation

We created `test_dataloader.py` to test the batching of training samples using PyTorch's DataLoader. The DataLoader retrieves multiple image-mask pairs together and prepares them for model training.

A batch size of four was used during testing. The successful test produced the following results:

- Image batch shape: `(4, 1, 512, 512)`
- Mask batch shape: `(4, 1, 512, 512)`
- Image data type: `torch.float32`
- Mask values: `0.0` and `1.0`
- Batch size: `4`

These results confirm that the dataset and DataLoader produce correctly shaped batches for the planned segmentation model.

11. Development Environment and Tools

The following tools and technologies were used during the completed work:

Tool or technology	Purpose
Python	Scripting and data processing

Visual Studio Code	Development environment
PyTorch	Tensor processing and dataset loading
NumPy	Numerical array operations
Pandas	Manifest CSV processing
Pillow	Image loading and mask generation
Matplotlib	Visualization of images and annotations
ARCADE dataset	Source of coronary angiography images and stenosis annotations

13. Next Steps

The following tasks remain to be completed:

- Image upload integration
- Uploaded-image preprocessing
- U-Net architecture
- Model training
- Model evaluation
- Prediction visualization
- System integration

14. **Status Summary**

The initial dataset analysis, stenosis annotation processing, binary mask generation, preprocessing, and PyTorch input pipeline have been completed. The successful DataLoader test confirms that the prepared image-mask pairs can be supplied to a deep learning model in the expected format.

Tasks: System Architecture, Database Schema, and UI Wireframes

1. Work Overview

As part of Milestone 1 of the project "Artery Segmentation and Stenosis Detection Using Advanced Deep Learning Techniques," we worked on designing the system architecture, preparing the database schema, and creating low-fidelity user interface wireframes for the proposed application, StenoVision.

Our work focused on defining the system structure, organizing application data, and establishing the initial user interface design to support future development.

2. System Architecture Design

We designed the high-level system architecture to identify the major components of StenoVision and illustrate how they interact. The architecture was organized into logical layers

covering the presentation interface, application services, computation, model processing, data storage, and infrastructure.

We also identified the supporting components required for security, monitoring, and external integrations. The architecture provides a reference for the implementation and integration activities planned in subsequent milestones.

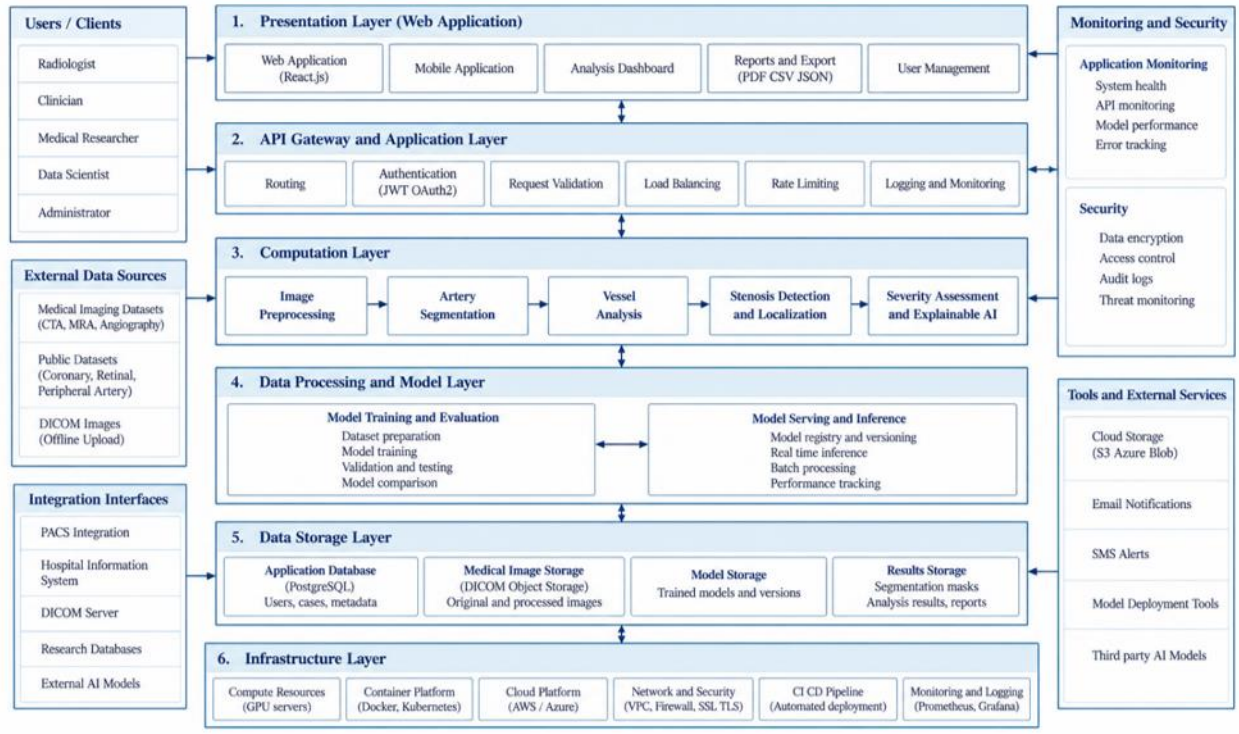
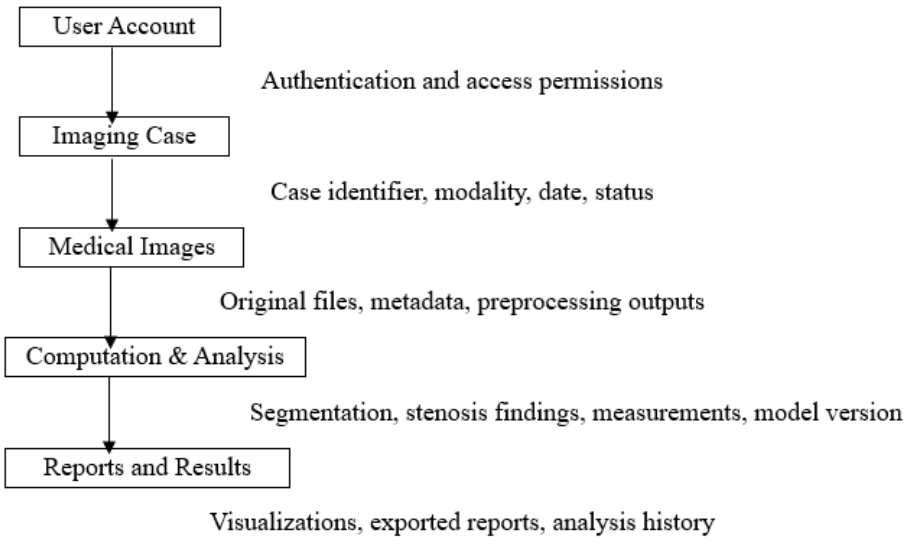


Figure 1: Proposed system architecture



3.Database Schema Design

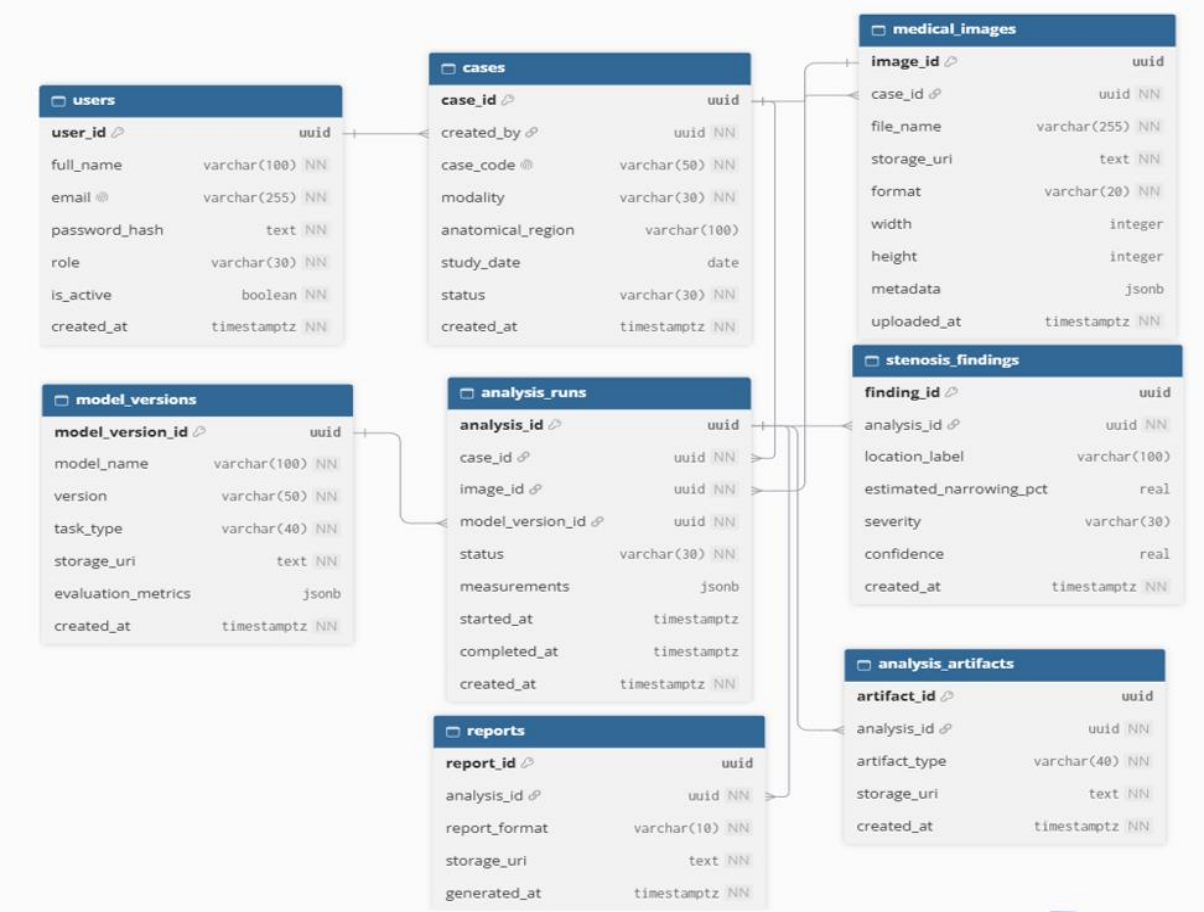


Figure 2: Proposed database schema and relationships between entities.

We prepared a proposed relational database schema to organize the information required for case management, medical image tracking, AI analysis, stenosis findings, and report generation.

We identified the following entities:

- users
- cases
- medical_images
- model_versions
- analysis_runs
- stenosis_findings
- analysis_artifacts
- reports

We defined primary keys, foreign keys, and relationships between these entities to establish a structured data model. The schema is designed to support case tracking, repeated analysis executions, model version tracking, and the storage of analysis results.

Dbdocs Link: ER Diagram

4. UI Wireframe Design

We created low-fidelity user interface wireframes using Figma to establish the proposed layout, navigation, and workflow of StenoVision. We maintained consistent header, sidebar navigation, grayscale styling, and structured screen layouts across the designs.

The following nine screens were designed:

1. Login
2. Dashboard
3. Case Management
4. Upload Medical Image
5. Analysis Configuration
6. Analysis Results
7. Case Details
8. Reports
9. User Management

These wireframes serve as visual references for the frontend development stage and establish the initial user experience for the application.

Figma Design Link:[UI wireframes]

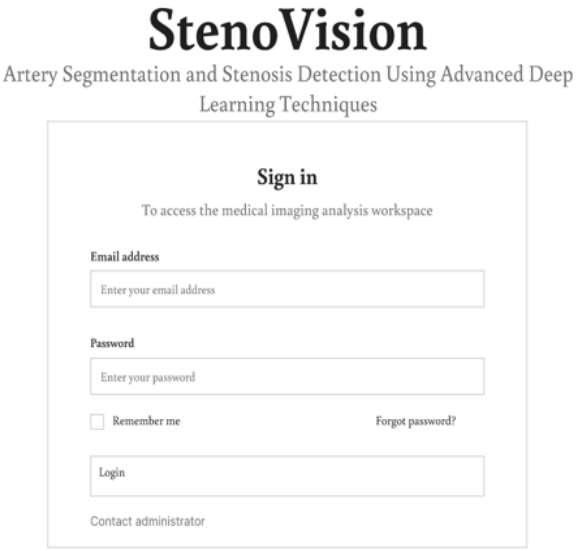


Figure 3: Registration and authentication.

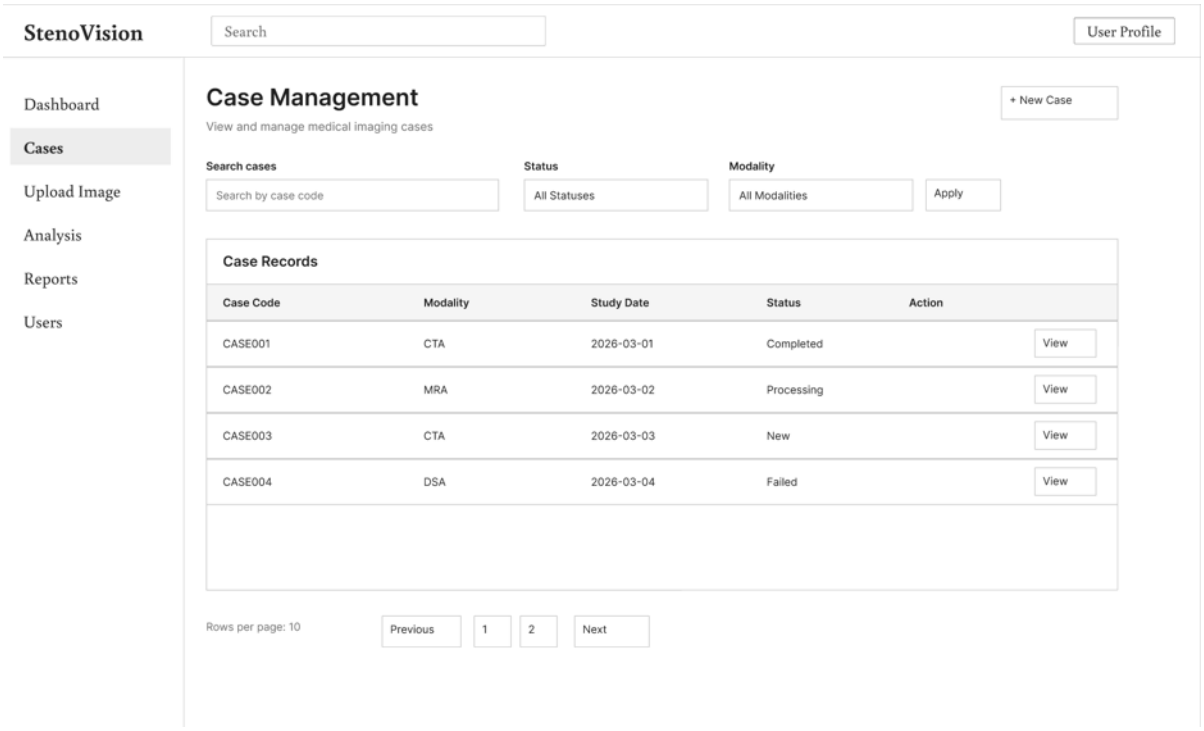


Figure 4: Dashboard and case management interface designs

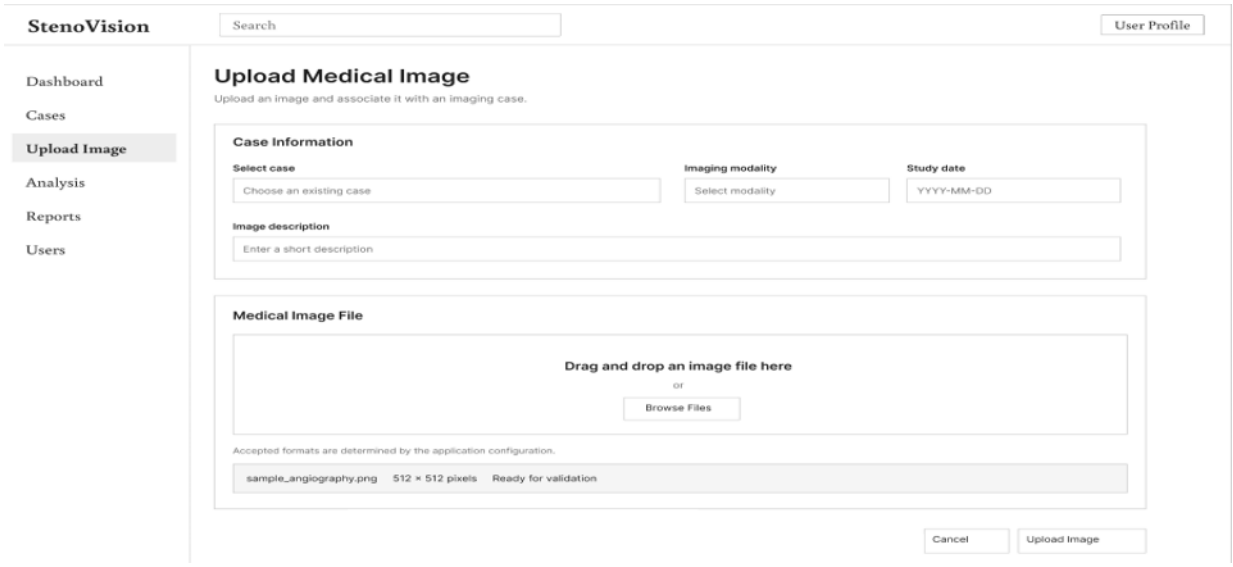


Figure 5: Medical image upload and analysis configuration interface designs.

5. Deliverables

The following deliverables were prepared during this milestone:

1. System architecture diagram.
2. Proposed database schema with defined entities and relationships.
3. Nine low-fidelity UI wireframes created in Figma.
4. Design references for the upcoming front-end development stage.

6. Next Steps

Based on the completed design work, we plan to proceed with frontend setup and initial interface implementation. The upcoming activities include configuring the development environment, creating reusable UI components, implementing navigation, and developing screens based on the Figma wireframes.

7. Status Summary

We have prepared the initial system architecture, database schema, and user interface wireframes for StenoVision. These deliverables establish the design foundation for the next phase of development and provide a reference for implementing the application's core functionality.

Team-3

Task Completed:

- **Set up backend.**
- **Implement authentication.**

StenoVision Backend: Initial Setup and Authentication Documentation

Module: Backend foundation (setup, registration, login, authentication, role-based access)

1. Purpose and Scope

This document describes the backend foundation of the StenoVision web application: how the project is organised, how it is configured and started, and how users register, log in and are authorised. Later modules, such as case management and segmentation, are built on top of it.

Included:

- **Project setup, configuration and database connection**
- **User registration and login**
- **Token-based authentication (JWT) and secure password storage**

- **Role-based access control (RBAC) for six roles**
- **Administrator functions for creating users and changing roles**
- **API routes, validation and error handling**

Not included (documented separately or planned):

- **Case management and image upload (see the Case Management document)**
- **Database migrations (see the Database Migrations section of that document)**
- **Segmentation and stenosis-detection models, the frontend, and deployment**

2. Technology Stack

- **Language: Python 3.11 or newer**
- **Web framework: FastAPI (REST endpoints, request validation, automatic Swagger documentation at /docs)**
- **Application server: Uvicorn (runs the FastAPI application)**
- **Database: PostgreSQL (local installation for development)**
- **ORM: SQLAlchemy 2.x (maps Python classes to database tables)**
- **Database driver: psycopg 3 (connects SQLAlchemy to PostgreSQL)**
- **Validation and configuration: Pydantic v2, pydantic-settings, email-validator**
- **Authentication: PyJWT (signed access tokens) and Passlib with bcrypt (password hashing)**
- **Form handling: python-multipart (reads the login form)**
- **Code quality: Ruff (linting and formatting)**
- **Version control: Git, with a branch-per-feature and pull-request review workflow**

3. System Architecture

The backend is layered. Each layer has one job, so a change in one place rarely breaks another.

Request flow: Client → Route (API layer) → Dependency (authentication and role check) → CRUD (database logic) → Model (database table). Schemas validate all data entering and leaving the API.

- **API routes (app/api/routes): HTTP endpoints. They stay thin and contain no database queries.**
- **Dependencies (app/api/deps.py): reusable checks that run before a route, such as getting the current user or requiring a role.**
- **Core (app/core): configuration and security helpers (hashing and tokens).**
- **CRUD (app/crud): all database queries.**
- **Models (app/models): database table definitions.**
- **Schemas (app/schemas): the shape of data accepted and returned by the API.**
- **Database (app/db): engine, session handling and first-run initialisation.**

Design principles:

- **Secrets and environment settings live in a .env file, never in code.**
- **Schemas are separate from models, so a password hash can never leak in a response.**
- **Access rules are enforced in one place (dependencies), not repeated in every route.**
- **The API is versioned under /api/v1, so a future version can be added without breaking the frontend.**

4. Project Structure

- **StenoVisionBackend/ (project root)**
  - **app/**
    - **main.py: application entry point; creates the app, runs startup initialisation, registers routers and the health check**
    - **api/**
- **deps.py: authentication and role-check dependencies**

- **routes/: auth.py (register, login, current user) and users.py (admin user management)**
- **core/: config.py (settings from the environment) and security.py (password hashing and tokens)**
- **crud/: user.py (user database operations)**
- **db/: session.py (engine, session, base class) and init_db.py (first admin creation)**
- **models/: user.py (User table and Role list) and __init__.py (registers all models)**
- **schemas/: user.py (request and response shapes)**
- **.env: local secrets and settings (not committed)**
- **.env.example: template with placeholder values (committed)**
- **.gitignore, requirements.txt, pyproject.toml (Ruff settings), Readme.md**
- **alembic.ini and migrations/: database migration setup**

Every folder inside app has an __init__.py file, which marks it as a Python package.

5. Dependencies

5.1 External packages

- **fastapi: the web framework (also installs Pydantic)**
- **uvicorn[standard]: the server that runs the application**
- **sqlalchemy: database access and table definitions**
- **psycopg[binary]: the PostgreSQL driver used by SQLAlchemy**
- **pydantic-settings: loads the .env file into typed settings**
- **email-validator: validates email addresses in schemas**
- **python-multipart: parses the login form and file uploads**
- **passlib[bcrypt]: password hashing framework with the bcrypt algorithm**
- **bcrypt (pinned to 4.0.1): newer versions are incompatible with Passlib**

- **PyJWT: creates and verifies signed tokens**
- **alembic: database migrations**
- **ruff: code linting and formatting**

5.2 Internal dependencies

- **main.py uses the routers, config and init_db.**
- **Routes use deps.py, crud, security, schemas and models.**
- **deps.py uses security, session and models.**
- **crud uses security (hashing) and models.**
- **security.py and session.py use config.**
- **init_db.py uses crud, session, models and config.**

Rule: routes call CRUD functions and never write database queries themselves.

6. Environment and Configuration

Settings are read from the .env file by app/core/config.py into one typed settings object. If a required value is missing, the application refuses to start.

- **SECRET_KEY (required): a long random string used to sign tokens. It is never shared or committed.**
- **DATABASE_URL (required): PostgreSQL connection string using the psycopg driver (user, password, host, port, database).**
- **ALGORITHM: token signing algorithm, default HS256.**
- **ACCESS_TOKEN_EXPIRE_MINUTES: token lifetime, default 30.**
- **FIRST_ADMIN_EMAIL and FIRST_ADMIN_PASSWORD: credentials of the first administrator, created at startup.**
- **PROJECT_NAME: title shown in the API documentation.**
- **STORAGE_ROOT and MAX_UPLOAD_MB: file storage settings used by the Case Management module.**

Rules: .env is excluded by .gitignore. .env.example lists every variable with fake values, so teammates know what to create. It must be updated whenever a variable is added.

7. Database Setup

- **Database: PostgreSQL installed locally. A dedicated database user and database are created using the psql shell, and each teammate runs their own local database.**
- **Connection and sessions (app/db/session.py): creates the engine (with a pre-ping check so dead connections are detected), a session factory, the Base class that all models inherit from, and the get_db function, which opens one session per request and always closes it afterwards.**
- **Schema management: tables are created and changed through Alembic migrations. The startup routine does not create tables.**
- **First administrator (app/db/init_db.py): at startup, the init_db function reads FIRST_ADMIN_EMAIL and FIRST_ADMIN_PASSWORD and creates an admin account if one does not exist. This is needed because public registration only creates patients, so without it nobody could assign roles.**

8. Data Model

Table: users (app/models/user.py)

- **id: unique identifier**
- **email: unique, indexed, stored in lowercase**
- **full_name: display name**
- **hashed_password: bcrypt hash (the plain password is never stored)**
- **role: one of the six roles below; defaults to patient**
- **is_active: when false, the user cannot log in or use existing tokens**
- **created_at: account creation timestamp**

Roles (Role list in the same file): admin, radiologist, clinician, doctor, patient, data_scientist.

9. Schemas (app/schemas/user.py)

- **UserCreate: email, full name (2 to 100 characters) and password (8 to 72 characters; 72 is bcrypt's limit). Used for public registration.**
- **AdminUserCreate: UserCreate plus a role. Used only by administrators.**
- **RoleUpdate: the new role for an existing user.**
- **UserRead: id, email, full name, role and active flag. It never includes the password hash.**
- **Token: the access token and its type (bearer).**

10. Security Design

10.1 Password storage

Passwords are hashed with bcrypt through Passlib (app/core/security.py). Hashing is one-way and salted, so two users with the same password have different hashes. At login, the entered password is checked against the stored hash. The original password is never stored or logged.

10.2 JSON Web Tokens

After a successful login the server returns a signed token containing the user id and an expiry time (30 minutes by default). The client sends it in the Authorization header with every request. Any change to the token breaks its signature, so the server can trust it without storing sessions.

The role is not stored in the token. On every request, the user and role are read from the database, so a role change or account deactivation takes effect immediately.

10.3 Key security decisions

- **Public registration always creates a patient. Staff roles can only be assigned by an administrator, which prevents anyone from registering as admin or doctor.**
- **Login errors do not reveal whether the email or the password was wrong.**
- **Emails are normalised to lowercase to prevent duplicate accounts.**
- **Secrets come only from environment variables.**
- **Deactivated users cannot log in or use existing tokens.**

11. Authentication Flow

Registration

1. The client sends email, full name and password.
2. Schema validation checks formats and lengths.
3. The server rejects an email that already exists (conflict).
4. The password is hashed and a patient account is saved.
5. The new user profile is returned without the hash.

Login

1. The client sends the email (in the username field) and password as a form.
2. The server finds the user and verifies the password.
3. Wrong credentials get an unauthorised response. A deactivated account gets a forbidden response.
4. On success, a signed token is returned.

Accessing a protected endpoint

1. The client sends the token in the Authorization header.
2. The token is verified and the user id is read from it.
3. The user is loaded from the database. Missing or deactivated users are rejected.
4. For restricted routes the role is checked. A wrong role gets a forbidden response.
5. The route runs.

12. Role-Based Access Control

- Admin: manages users and roles; full access to the system.
- Radiologist: works with imaging cases and uploads scans.
- Clinician: works with and reviews patient cases.
- Doctor: reviews cases and results for treatment.
- Patient: sees only their own information.

- **Data scientist: reads the data needed for model development, without write access.**

  How enforcement works: the require_roles function in app/api/deps.py builds a check from a list of allowed roles. Protecting any endpoint takes one line. The status codes mean: 401, not signed in; 403, signed in but not permitted.

  Current use: the user management endpoints are admin only. The case module applies its own role rules through a permissions file (see the Case Management document).

  13. API Reference

  Base path: /api/v1. Interactive documentation is available at /docs.

  Authentication (app/api/routes/auth.py)

- **POST /auth/register**
  - **Access: public**
  - **Input: email, full name, password**
  - **Result: the new patient profile (201)**
  - **Errors: 409 email already registered; 422 invalid input**
- **POST /auth/login**
  - **Access: public**
  - **Input: form with username (the email) and password**
  - **Result: access token and token type**
  - **Errors: 401 incorrect email or password; 403 account deactivated**
- **GET /auth/me**
  - **Access: any signed-in user**
  - **Result: the current user's profile**
  - **Errors: 401 missing, invalid or expired token**

Summary:

1. Basic backend setup completed.

2. Login and Register modules based on RBAC completed

3. Authentication and secure routing of API endpoints complete.

Next Steps:

1. **Complete the partially completed case management backend system.**
2. **Integrate frontend and backend API to demonstrate project work.**

Team-4

- **Set up frontend .**

  9 Screens are in pages/ folder:

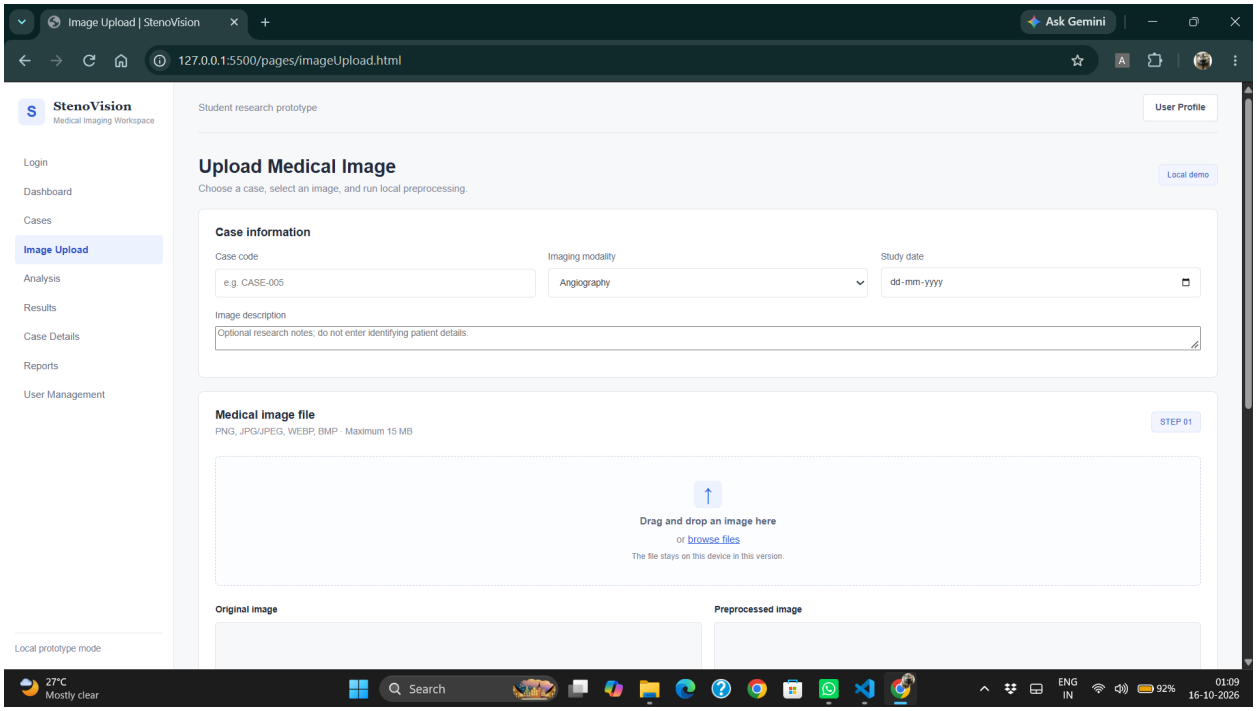
  Login, Dashboard, Case Management, Upload Image, Analysis Configuration, Analysis Results, Case Details, Reports, User Management All pages are standalone HTML. Shared logic is in: css/styles.css - all styling js/common.js - This is the main brain. It handles:

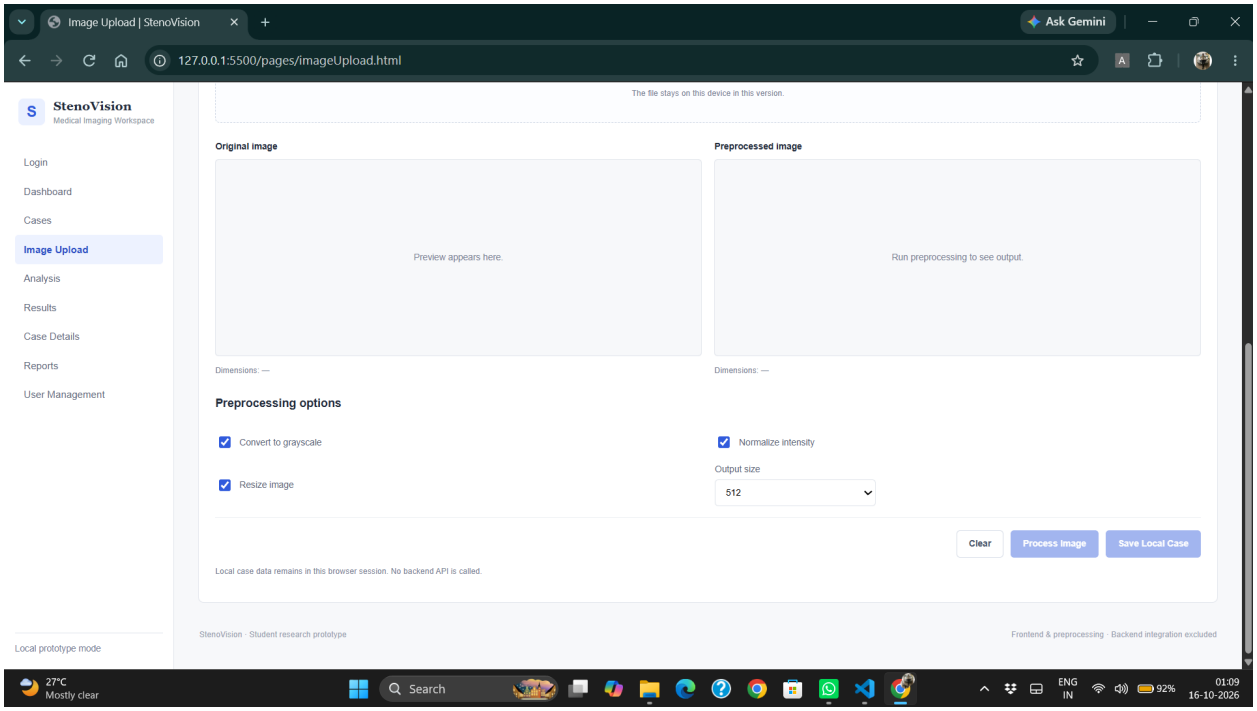
  Generate the sidebar navigation on every page. Local case metadata in local Storage Image validation / preview / browser preprocessing Tables and CSV export

  No backend request is made. If you clear the browser cache, case data will be gone.

- **Implement image upload**

  Here is what the Upload Image section does:
  1. Purpose:
  To select an image for a case, validate it, preview it, and do browser-side preprocessing before analysis.
  2. Features included (from js/common.js): Image Selection: User can browse and select an image.
  Validation: Checks file type and size Supported: PNG, JPG/JPEG, WEBP, BMP only Not Supported: DICOM / NIfTI in frontend Max Size: 15 MB
  Preview: Shows image preview after selection.
  Browser-side Preprocessing: Basic preprocessing happens in the browser itself (handled in js/common.js), no server call.
  Local Python Pipeline: Separate local preprocessing script also available in preprocessing/preprocess.py for resizing to 512x512 etc.
  Case Linking: Upload is linked to case metadata which is saved in browser local Storage.





- **Build initial preprocessing pipeline.**

Milestone 2 (Anwesha Singh)

Task 1: Implement Baseline Segmentation Model

What it means

A baseline is the simplest standard model that solves the problem. For medical image segmentation, that is the **U-Net**. It is not expected to be the best model. Its purpose is to give a reference score that every advanced model must beat.

Why it is needed

Without a baseline you cannot prove that a complex model actually helps. If the advanced model only matches the baseline, the simpler and faster model should be preferred.

How U-Net works

- **Encoder:** repeated convolution and pooling layers that shrink the image while learning richer features (edges, then vessel shapes).
- **Decoder:** upsampling layers that rebuild the image size and produce a mask.
- **Skip connections:** feature maps copied from encoder to decoder so fine details (thin vessel boundaries) are not lost.
- **Output:** one channel of probabilities per pixel. A threshold of 0.5 turns it into a binary mask.

Implementation steps

1. Build the dataset class: read image, convert polygons to a mask, resize both, apply CLAHE to the image.
2. Define a U-Net with a 1-channel grayscale input and 1-channel output (base width 32, four down-sampling levels).
3. Choose the loss: **BCE + Dice loss**. Vessels cover only a small part of each image, so plain accuracy would reward predicting all background.
4. Choose the optimizer and schedule: AdamW, learning rate 1e-3, halve the rate when validation Dice stalls.
5. Evaluate with Dice, IoU, precision and recall.

Deliverable

A working U-Net, its code, and its Dice and IoU on validation and test sets.

Common problems

- Output is all black (all background): increase Dice loss weight, lower the threshold, or check that masks are not empty.
- Image size not divisible by 16 causes shape errors in the custom U-Net.
- Image and mask misaligned after augmentation: apply the same transform to both and use nearest-neighbour interpolation for masks.

Task 2: Implement Advanced Segmentation Architecture

What it means

Implement at least one stronger architecture and test whether it improves on the baseline. The project brief lists U-Net++, Attention U-Net, ResUNet, DeepLabV3+, nnU-Net, TransUNet and Swin-UNet as candidates. A practical choice for a student project is two of them.

Implementation steps

1. Implement the chosen architecture (custom code or the `segmentation-models-pytorch` library).
2. Keep everything else identical to the baseline: same split, preprocessing, augmentation, loss, optimizer and epochs.
3. Record number of parameters and inference time for each model.

Deliverable

One or two advanced models, trained under the same conditions, with their metrics.

Task 3: Train Segmentation Models

What it means

Training is the process where the model learns from the 1,000 training images by repeatedly predicting masks, measuring the error and adjusting its weights.

Data splits

- **Train (1,000):** the model learns from these.

- **Validation (200):** used after every epoch to monitor progress, tune settings and select the best checkpoint.
- **Test (300):** used **once at the end** for the final unbiased score.

What to save

- Best checkpoint (by validation Dice) for every model, saved to Google Drive.
- Training history: loss and Dice per epoch.
- The exact settings used, so results are reproducible.

How to read the training curves

- Training loss falls but validation loss rises: **overfitting**. Use more augmentation, a smaller model, or stop earlier.
- Both stay high: **underfitting**. Train longer, use a stronger model, or check labels.
- Validation Dice jumps around: use a smaller learning rate or a larger batch.

Deliverable

Trained models, saved checkpoints and training-curve plots.

Task 4: Generate Artery Masks

What it means

Use the best trained model to produce an artery mask for every image, and save the masks as files so the later steps (post-processing, centerline, diameter) can use them. This is the inference stage, and the same code will later sit behind the FastAPI upload endpoint.

Steps

1. Load the best checkpoint.
2. For each image: apply the same preprocessing as training (resize, CLAHE, scale to 0-1).
3. Run the model to get a probability map (values 0 to 1).
4. Threshold at 0.5 to get the binary mask.
5. Save the **probability map**, the **binary mask** (PNG, values 0 and 255) and an **overlay image** (mask drawn over the angiogram) for each case.
6. Name outputs after the source image so cases can be traced, and keep masks at the original image size if later measurements must match the original pixels.

Why save the probability map too

It lets you try different thresholds later and gives a confidence value for the explainability and alert modules.

Deliverable

A folder of predicted masks and overlays for the validation and test images, plus a function `segment_image(image)` that works on a single new image.

Task 5: Implement Segmentation Post-processing

What it means

Raw model output is often imperfect: isolated specks, small holes inside vessels, and breaks along thin vessels. Post-processing cleans the mask using classical image-processing rules.

How to decide the settings

Tune threshold, minimum size and closing radius **on the validation set only**. Measure Dice before and after post-processing. Keep an operation only if it improves (or at least does not reduce) the score. Report both numbers.

Caution

Over-aggressive cleaning can delete real thin vessels or close the very narrowing you want to detect later. Always check overlays by eye, especially around thin branches.

Deliverable

A post-processing function, validation Dice before vs after, and example before/after images.

Task 6: Extract Vessel Structures

What it means

Turn the clean mask from a picture into a **structure**: which vessels exist, how they are connected, and where they branch. Stenosis analysis needs to work vessel by vessel, not on a raw mask.

What to extract

- **Connected components:** count and size of separate vessel trees (for example `cv2.connectedComponentsWithStats`).
- **Vessel segments:** pieces of the skeleton between two key points.
- **Key points:** branch points (where a vessel splits) and end points (where a vessel stops).
- **Region of interest:** bounding box of each vessel structure.
- **Vessel labels (optional):** ARCADE `syntax` segment labels can be used to name segments (for example proximal, mid and distal parts of the main arteries), if you later move to multi-class segmentation.

Steps

1. Take the post-processed mask.
2. Label connected components and discard tiny ones.
3. Compute the skeleton (Task 7) and classify skeleton pixels: a pixel with one neighbour is an end point, with three or more neighbours is a branch point.
4. Remove the branch points to split the skeleton into separate segments, then give each segment an ID.
5. Store the structure as a simple table: segment ID, start point, end point, length in pixels, parent or branch point.

Deliverable

A vessel-structure table or JSON per image, and an image showing segments, branch points and end points in different markers.

Task 7: Implement Centerline Extraction

What it means

The centerline (skeleton) reduces each vessel to a thin line, one pixel wide, running along its middle. It gives the path along which length, tortuosity and diameter are measured.

Cleaning the skeleton

- **Fix gaps:** if a vessel is broken in the mask, the skeleton will be broken too. Closing in post-processing helps.

- **Order the points:** for each segment, order the pixels from one end to the other so a profile can be plotted along it.

Quantities that come from the centerline

- Vessel length (sum of distances between consecutive pixels; diagonal steps count as 1.41).
- Number of branches and branch points.
- **Tortuosity:** path length divided by the straight-line distance between the ends.

Deliverable

A centerline image overlaid on the angiogram, plus branch and end points marked.

Task 8: Estimate Vessel Diameter

What it means

Measure how wide the vessel is at every point along its centerline. A sudden drop in diameter is the signal for stenosis, so this is the most important input for Milestone 3.

Per-vessel diameter profile

For each ordered segment, record the diameter at every centerline point and compute:

- mean diameter,
- minimum diameter and where it occurs,
- local diameter variation (for example the drop from a moving average),
- cross-sectional area (for the 2D projection, vessel area along the segment).

Smooth the profile slightly (moving average or Savitzky-Golay filter) so pixel noise does not look like narrowing.

Important limitations to state

- ARCADE images are PNGs with **no pixel spacing**, so diameters are in **pixels**, not millimetres. Percentage narrowing is still valid because it is a ratio.
- X-ray angiography is a 2D projection, so foreshortening and overlapping vessels can distort widths.
- Accuracy depends on mask quality. A mask that is a pixel too thin or too thick changes the diameter noticeably for small vessels.

Task 9: Compare Segmentation Models

What it means

Show, with numbers, which model is best and why. This is how the milestone requirement "best-performing segmentation model selected" is met.

Fair comparison rules

- Same data split, preprocessing, augmentation, loss, optimizer, image size and seed for all models.
- Select the best checkpoint using **validation** Dice.
- Report final numbers on the **test** set, evaluated once.

Extra analysis that strengthens the comparison

- Per-image Dice distribution (box plot) to show consistency across test cases.
- Boundary metrics: Hausdorff distance and average surface distance, since the brief lists them.
- Visual comparison on the same difficult images (thin vessels, low contrast).
- Effect of post-processing on each model.
- Trade-off between accuracy, model size and speed. A slightly better model that is much slower may not suit the web application.

Decision

Choose the model that gives the best validation and test Dice, with consistent per-image results and acceptable speed. If the baseline is close or better, say so honestly and choose the simpler model.

Deliverable

The comparison table, a chart, overlay comparisons and a written conclusion naming the selected model.

Task 10: Generate Segmentation Performance Reports

What it means

Summarise the whole segmentation stage in a clear report that your mentor can read without running any code. It also feeds the dashboard and report-generation modules later.

Suggested report structure

1. **Objective and data:** task, ARCADE dataset, split sizes, binary mask definition.
2. **Method:** preprocessing, augmentation, models, loss, optimizer, hardware.
3. **Training results:** loss and Dice curves for each model.
4. **Quantitative results:** comparison table with all metrics, and per-image distribution.
5. **Qualitative results:** overlays of good, average and failure cases.
6. **Post-processing effect:** Dice before and after.
7. **Vessel analysis samples:** centerline and diameter profile examples.
8. **Failure analysis:** where models fail (thin distal vessels, low contrast, overlapping vessels, catheter or bone shadows) and why.
9. **Limitations:** binary mask only, polygon labels are approximate, no pixel spacing, single dataset.

Conclusion and next step: selected model and how it feeds stenosis detection in Milestone 3.

Output files

- PDF or Word report for the mentor.
- CSV of per-image metrics.
- JSON summary of final metrics (later reused by the dashboard).
- Saved plots (training curves, comparison chart, overlays).